

L27: Independent Replications and Confidence Intervals

Simulation Without a Black Box

Outline

Why One Run Is Not Enough

Independent Replications

Confidence Intervals

Sample Size via Pilot Study

Coverage and Batch Means

Key Takeaways

Why One Run Is Not Enough

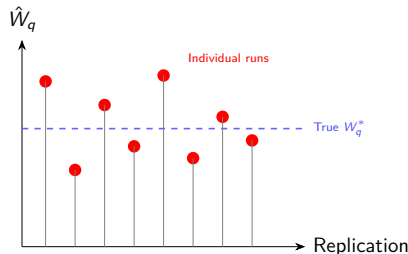
The Fundamental Problem

A single simulation run produces one realization of a stochastic process. It is an *observation*, not an *estimate* with quantified uncertainty.

Analogy: Asking “what is the average human height?” and measuring one person. You get a number, but you have no idea how accurate it is.

Consequences of reporting a single run:

- No way to know whether a 5-minute difference in W_q between designs A and B is signal or noise
- Confidence intervals require variance estimates, which require multiple observations
- Sample-to-sample variability can easily exceed the effect size of interest



Independent Replications with Seeded RNG

Key Requirement

Replications must be *statistically independent*.
Use a separate random seed for each — never reuse or continue from the previous run's state.

NumPy's `rng.spawn(n)` creates n independent child generators from a parent, guaranteed to produce non-overlapping streams (SeedSequence algorithm).

- Output: $\hat{\theta}_1, \hat{\theta}_2, \dots, \hat{\theta}_R$ — one estimate per replication
- Treat these as i.i.d. draws from the sampling distribution of $\hat{\theta}$
- Apply standard one-sample t inference

```
import numpy as np

def run_one(seed):
    """Returns Wq estimate for one rep."""
    rng = np.random.default_rng(seed)
    # ... SimPy clinic model here ...
    return wq_estimate

# Spawn R independent seeds
parent = np.random.SeedSequence(12345)
seeds = parent.spawn(30)

results = np.array([
    run_one(s) for s in seeds
])

print(f"Replications: {len(results)}")
print(f"Mean Wq: {results.mean():.2f}")
print(f"Std dev: {results.std(ddof=1):.2f}")
```

The 95% Confidence Interval

CI Formula

$$\bar{X} \pm t_{n-1, 0.025} \cdot \frac{s}{\sqrt{n}}$$

where $\bar{X}$ = sample mean, s = sample std, n = replications.

Correct interpretation: “If we ran this experiment many times, 95% of the resulting intervals would contain the true θ .”

Common mistake: “There is a 95% probability that θ lies in this specific interval.” — The interval is fixed after data are observed; θ is not random.

Warning

Always report both $\bar{X}$ and the half-width h . A CI without a point estimate is incomplete. A point estimate without a CI is misleading.

```
import numpy as np, scipy.stats as st

n      = len(results)
xbar   = results.mean()
s      = results.std(ddof=1)

# t critical value, 2-tailed, 95%
t_crit = st.t.ppf(0.975, df=n-1)
hw      = t_crit * s / np.sqrt(n)

print(f"n          = {n}")
print(f"Mean Wq      = {xbar:.2f} min")
print(f"Std dev       = {s:.2f} min")
print(f"Half-width=    {hw:.2f} min")
print(f"95% CI        = [{xbar-hw:.2f}, "
      f" {xbar+hw:.2f}]")
```

Pilot Study and Sequential Sample-Size Formula

Goal: achieve half-width $\leq h^*$ (the operationally meaningful precision level).

Sequential procedure:

1. Run a pilot of $n_0 \geq 10$ replications; compute s_0 .
2. Compute required sample size:

$$n^* = \left\lceil \left(\frac{t_{n_0-1, 0.025} \cdot s_0}{h^*} \right)^2 \right\rceil$$

3. Run $\max(n^* - n_0, 0)$ additional replications.
4. Compute final CI from all n^* results.

Warning

n^* depends on s_0 , which is itself random. The formula gives a *target*, not a guarantee. If the final half-width still exceeds h^* , run more replications.

```
import numpy as np, scipy.stats as st

# Pilot study
n0 = 10
pilot = np.array([run_one(s)
                  for s in seeds[:n0]])
s0 = pilot.std(ddof=1)
h_star = 1.0 # target half-width (min)

t_pilot = st.t.ppf(0.975, df=n0-1)
n_star = int(np.ceil(
    (t_pilot * s0 / h_star)**2))

print(f"Pilot std = {s0:.2f}")
print(f"Required n = {n_star}")

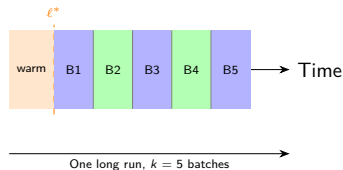
# Run additional reps if needed
extra = max(n_star - n0, 0)
more_seeds = parent.spawn(extra + n0)
all_results = np.array([
    run_one(s) for s in more_seeds[:n_star]
])
```

Coverage, Batch Means, and Choosing a Strategy

Coverage: A 95% CI has *coverage* if it contains the true θ . Across many experiments, a correctly constructed CI achieves coverage $\rightarrow 95\%$ as $n \rightarrow \infty$.

Batch means (alternative to replications):

- Run one very long simulation
- Divide post-warmup output into k non-overlapping batches
- Batch means are approximately i.i.d. if batches are long enough
- Apply same t formula to batch means



Method	Pro	Con
Replications	Exact independence	Startup cost repeated
Batch means	One long run	Autocorrelation if batches too short

Key Takeaways

1. **One run is an observation, not an estimate** — always quantify uncertainty with multiple replications.
2. **Use `rng.spawn(n)`** to generate independent seeds; never reuse or continue a single stream.
3. **95% CI:** $\bar{X} \pm t_{n-1,0.025} \cdot s/\sqrt{n}$; report both point estimate and half-width.
4. **Pilot study formula:** $n^* = \lceil (t \cdot s_0/h^*)^2 \rceil$ — a target, not a guarantee; re-check after collecting data.
5. **Coverage = 95%** means 95 of 100 intervals cover θ , not a statement about any single interval.
6. **Batch means** are useful for very long steady-state runs but require sufficiently long batches to break autocorrelation.

Next Lecture

L28 — Common random numbers for variance reduction in scenario comparison.